

FLIPTECH

Jeu de cartes – Document technique

Table des matières

1	Introduction	3
1.1	Présentation du projet	3
1.2	Objectifs pédagogiques	3
1.3	Principe général du jeu	3
1.4	Composition du jeu de cartes	3
1.4.1	Cartes numérotées (79 cartes)	4
1.4.2	Cartes bonus (6 cartes)	4
1.5	Déroulement détaillé	4
1.5.1	Déroulement d'une manche	4
1.5.2	Calcul du score d'une manche	4
1.5.3	Fin de partie	5
2	Contraintes techniques	6
2.1	Normes de qualité	6
2.2	Architecture obligatoire	6
2.3	Dépendances entre modules	7
2.4	Validation des entrées	7
3	Répartition des tâches	8
3.1	Équipe de développement	8
3.2	Détail des responsabilités	8
3.2.1	Brahim – Module Deck & Utilitaires	8
3.2.2	Kais – Module Moteur & Règles	8
3.2.3	Youssef – Module Interface & Persistance	8
3.3	Matrice des responsabilités	9
3.4	Planning et jalons	9
3.5	Recommandations pour l'équipe	9
4	Partie 1 : Gestion du deck	10
4.1	Exercice 1.1 : Definition des structures	10
4.2	Exercice 1.2 : Generation exacte du deck	10
4.3	Exercice 1.3 : Melange du deck	11
4.4	Exercice 1.4 : Pioche securisee	11
4.5	Exercice 1.5 : Fonctions utilitaires (optionnel mais recommande)	12

5	Partie 2 : Moteur du jeu	13
5.1	Exercice 2.1 : Structure Player	13
5.2	Exercice 2.2 : Application des regles	14
5.3	Exercice 2.3 : Gestion des manches	15
5.4	Exercice 2.4 : Boucle de jeu principale	15
5.5	Exercice 2.5 : Intelligence artificielle (bonus)	16
6	Partie 3 : Interface utilisateur	17
6.1	Exercice 3.1 : Affichage des cartes et des joueurs	17
6.2	Exercice 3.2 : Saisie securisee	18
6.3	Exercice 3.3 : Sauvegarde des scores	18
6.4	Exercice 3.4 : Statistiques	19
6.5	Exercice 3.5 : Cartes speciales (bonus)	19
7	Partie 4 : Integration, tests et rendu final	20
7.1	Exercice 4.1 : Programme principal	20
7.2	Exercice 4.2 : Liste des tests obligatoires	20
7.3	Exercice 4.3 : Controles de qualite finaux	21
7.4	Contenu du livrable	21
7.5	Barème indicatif	21
7.6	Conseil final	21

1 Introduction

1.1 Présentation du projet

FlipTech est un jeu de cartes stratégique où les joueurs accumulent des points en piochant des cartes, tout en évitant les doublons. Le projet s'inscrit dans le cadre du cours **Projet Informatique 2** de la filière preING1.

L'objectif est de développer une implémentation complète du jeu en langage C, en respectant une architecture modulaire stricte et des contraintes de qualité industrielle.

1.2 Objectifs pédagogiques

Ce projet vise à développer les compétences suivantes :

- Maîtrise du langage C (pointeurs, structures, gestion mémoire)
- Conception et implémentation d'une architecture modulaire
- Travail collaboratif avec séparation des responsabilités
- Gestion des erreurs et robustesse du logiciel
- Respect des normes de codage et documentation

1.3 Principe général du jeu

Rappel des règles

Le but du jeu est d'obtenir le plus de points possible. Une partie se compose de plusieurs manches.

Une manche se termine lorsque :

- Un joueur obtient 7 cartes différentes
- Tous les joueurs décident de sortir volontairement
- Un joueur pioche une carte qu'il possède déjà (doublon)

La partie se termine lorsqu'un joueur atteint ou dépasse **200 points**, ou lorsque la pioche est vide.

1.4 Composition du jeu de cartes

Le jeu contient exactement **85 cartes** réparties comme suit :

1.4.1 Cartes numérotées (79 cartes)

Valeur	Quantité
0 – 1	1 carte chacun
2	2 cartes
3	3 cartes
4	4 cartes
5	5 cartes
6	6 cartes
7	7 cartes
8	8 cartes
9	9 cartes
10	10 cartes
11	11 cartes
12	12 cartes

1.4.2 Cartes bonus (6 cartes)

Carte	Effet	Quantité
x2	Multiplie le score de la manche par 2	1
+2	Ajoute 2 points au score final de la manche	1
+4	Ajoute 4 points au score final de la manche	1
+6	Ajoute 6 points au score final de la manche	1
+8	Ajoute 8 points au score final de la manche	1
+10	Ajoute 10 points au score final de la manche	1

1.5 Déroulement détaillé

1.5.1 Déroulement d'une manche

1. Les joueurs jouent dans un ordre fixe, déterminé en début de partie
2. À chaque tour, un joueur a le choix entre :
 - **Sortir** : il conserve son score actuel pour cette manche
 - **Piocher** : il tire une carte du deck
3. En cas de pioche, quatre situations sont possibles :
 - (a) **Carte numéro nouvelle** : la valeur de la carte est ajoutée au score de manche
 - (b) **Carte bonus** : l'effet est mémorisé et s'applique en fin de manche
 - (c) **Doublon** : le joueur perd immédiatement la manche (score = 0)
 - (d) **7 cartes différentes** : la manche s'arrête immédiatement, le joueur reçoit un bonus de +15 points

1.5.2 Calcul du score d'une manche

Le score final d'une manche est calculé selon la formule :

$$\text{Score} = (\text{Somme des cartes numérotées}) \times (\text{Multiplicateur bonus}) + (\text{Addition bonus})$$

Les bonus s'accumulent pendant la manche et s'appliquent uniquement si le joueur n'a pas perdu.

1.5.3 Fin de partie

La partie s'arrête dès qu'au moins une de ces conditions est remplie :

- Un joueur atteint ou dépasse 200 points cumulés
- La pioche est épuisée (plus aucune carte à tirer)

Le vainqueur est le joueur avec le score total le plus élevé.

2 Contraintes techniques

2.1 Normes de qualité

Attention

- **Zéro crash** : le programme ne doit jamais planter (segmentation fault, boucle infinie)
- **Pas de variables globales** : toute donnée partagée passe par paramètres
- **Limite de 40 lignes par fonction** : une fonction doit faire une seule chose
- **Limite de 300 lignes par fichier source** : favorise la modularité
- **Code commenté** : l'anglais est recommandé pour les commentaires
- **Séparation stricte** : logique métier et affichage doivent être dans des modules distincts

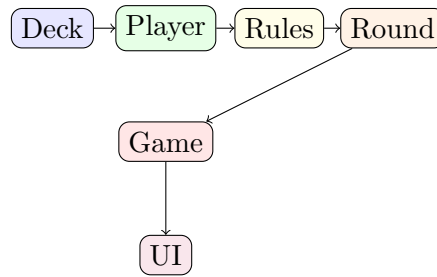
2.2 Architecture obligatoire

Le projet doit respecter l'arborescence suivante :

```
fliptech/  
|-- src/  
|   |-- main.c  
|   |-- deck/  
|   |   |-- deck.c  
|   |   |-- deck.h  
|   |-- game/  
|   |   |-- game.c  
|   |   |-- game.h  
|   |   |-- round.c  
|   |   |-- round.h  
|   |-- player/  
|   |   |-- player.c  
|   |   |-- player.h  
|   |-- rules/  
|   |   |-- rules.c  
|   |   |-- rules.h  
|   |-- ui/  
|   |   |-- display.c  
|   |   |-- display.h  
|   |   |-- input.c  
|   |   |-- input.h  
|   |-- save/  
|   |   |-- save.c  
|   |   |-- save.h  
|   |-- stats/  
|   |   |-- stats.c  
|   |   |-- stats.h  
|   |-- utils/  
|   |   |-- random.c  
|   |   |-- random.h  
|-- include/  
|-- data/  
|-- Makefile  
|-- README.md
```

2.3 Dépendances entre modules

L'ordre d'intégration est impératif :



2.4 Validation des entrées

Toutes les saisies utilisateur doivent être sécurisées :

- Refus des valeurs hors bornes
- Gestion des erreurs de frappe (lettres au lieu de chiffres)
- Buffer propre après chaque saisie erronée
- Redemande systématique jusqu'à obtention d'une valeur valide

3 Répartition des tâches

3.1 Équipe de développement

Membre	Rôle principal	Second rôle
Brahim	Backend	Deck & randomisation
Kais	Game Design	Moteur & règles
Youssef	Frontend	Interface & persistance

3.2 Détail des responsabilités

3.2.1 Brahim – Module Deck & Utilitaires

Partie réservée à Brahim

- Définition des structures Card et Deck
- Génération exacte du jeu de 85 cartes
- Algorithme de mélange (Fisher-Yates)
- Pioche sécurisée avec gestion du deck vide
- Fonctions utilitaires (deck_print, deck_is_empty, deck_size)
- Générateur aléatoire (random.c / random.h)

3.2.2 Kais – Module Moteur & Règles

Partie réservée à Kais

- Structure Player et gestion de la main
- Détection des doublons et des 7 cartes différentes
- Application des règles de jeu (bonus, scores)
- Gestion des manches (reset, fin de manche)
- Boucle principale du jeu
- Intelligence artificielle basique (bonus)
- Coordination entre les modules

3.2.3 Youssef – Module Interface & Persistance

Partie réservée à Youssef

- Affichage couleur des cartes et des joueurs
- Saisies utilisateur sécurisées
- Menu principal et navigation
- Sauvegarde des scores dans des fichiers
- Statistiques (meilleur joueur, moyenne, classement)
- Interface IA (affichage des décisions)
- Gestion des erreurs utilisateur

3.3 Matrice des responsabilités

Fichier	Brahim	Kais	Youssef
deck.c / deck.h	✓	–	–
random.c / random.h	✓	–	–
player.c / player.h	–	✓	–
rules.c / rules.h	–	✓	–
round.c / round.h	–	✓	–
game.c / game.h	–	✓	–
display.c / display.h	–	–	✓
input.c / input.h	–	–	✓
save.c / save.h	–	–	✓
stats.c / stats.h	–	–	✓
main.c	–	–	✓

3.4 Planning et jalons

Jalon	Date	Livrables
J1 – Deck	S+1	deck.c, deck.h, random.c, random.h
J2 – Player & Rules	S+2	player.c, rules.c, tests unitaires
J3 – Round & Game	S+3	round.c, game.c, intégration partielle
J4 – UI & Sauvegarde	S+4	display.c, input.c, save.c, stats.c
J5 – Intégration finale	S+5	main.c, Makefile, README, Rapport

3.5 Recommandations pour l'équipe

Attention

Bonnes pratiques de collaboration :

- Utiliser **Git** avec des branches par fonctionnalité
- Faire des **revues de code** avant fusion
- Documenter les fonctions dans les fichiers .h
- Tester chaque module indépendamment avant intégration
- Communiquer quotidiennement sur l'avancement

4 Partie 1 : Gestion du deck

Partie réservée à Brahim

Cette partie est entièrement réservée à Brahim. Les fichiers concernés sont : deck.c, deck.h, random.c, random.h.

4.1 Exercice 1.1 : Définition des structures

1. Ouvrez le fichier deck.h.
2. Déclarez un type énuméré nommé CardType qui contiendra trois valeurs : NUM, BONUS_MULT, BONUS_ADD.

```
typedef enum {  
    NUM,  
    BONUS_MULT,  
    BONUS_ADD  
} CardType;
```

3. Déclarez une structure Card qui contiendra deux champs :
 - type : de type CardType
 - value : de type int
4. Déclarez une structure Deck qui contiendra deux champs :
 - cards : un tableau de 85 Card
 - size : un entier représentant le nombre de cartes restantes dans la pioche
5. Ajoutez les gardes d'inclusion (#ifndef, #define, #endif) pour éviter les inclusions multiples.
6. Compilez avec la commande gcc -c deck.h pour vérifier l'absence d'erreurs de syntaxe.

4.2 Exercice 1.2 : Génération exacte du deck

1. Ouvrez le fichier deck.c.
2. Créez la fonction suivante :

```
void deck_init(Deck *deck);
```

3. Dans cette fonction, vous devez :
 - (a) Initialiser deck->size à 0.
 - (b) Ajouter les cartes numéros. Pour cela, utilisez une boucle for qui parcourt les valeurs de 0 à 12. Pour chaque valeur, le nombre d'occurrences est :
 - valeur 0 : 1 occurrence
 - valeur 1 : 1 occurrence
 - valeur 2 : 2 occurrences
 - valeur 3 : 3 occurrences
 - valeur 4 : 4 occurrences
 - valeur 5 : 5 occurrences
 - valeur 6 : 6 occurrences
 - valeur 7 : 7 occurrences
 - valeur 8 : 8 occurrences
 - valeur 9 : 9 occurrences
 - valeur 10 : 10 occurrences

- valeur 11 : 11 occurrences
- valeur 12 : 12 occurrences

Pour chaque occurrence, creez une carte de type NUM avec la valeur correspondante, stockez-la dans `deck->cards[deck->size]`, puis incrementez `deck->size`.

- (c) Ajouter les 6 cartes bonus dans l'ordre suivant :
- x2 : type BONUS_MULT, valeur 2
 - +2 : type BONUS_ADD, valeur 2
 - +4 : type BONUS_ADD, valeur 4
 - +6 : type BONUS_ADD, valeur 6
 - +8 : type BONUS_ADD, valeur 8
 - +10 : type BONUS_ADD, valeur 10
4. Creez un fichier temporaire `test_deck.c` avec une fonction main pour tester votre code :
- (a) Declarez une variable Deck `deck`.
 - (b) Appelez `deck_init(&deck)`.
 - (c) Affichez `deck.size`. La valeur doit etre 85.
 - (d) Parcourez le tableau de cartes et affichez chaque carte (type et valeur) pour verification visuelle.
5. Une fois la validation effectuee, commentez ou supprimez les affichages de test.

4.3 Exercice 1.3 : Melange du deck

1. Dans `deck.c`, ajoutez les inclusions necessaires :

```
#include <stdlib.h>
#include <time.h>
```

2. Creez la fonction suivante :

```
void deck_shuffle(Deck *deck);
```

3. Implementez l'algorithme de Fisher-Yates :
- (a) Initialisez le generateur aleatoire avec `srand(time(NULL))`.
 - (b) Parcourez `i` de `deck->size - 1` jusqu'a 1 (decrement).
 - (c) Pour chaque `i`, calculez un indice `j` aleatoire entre 0 et `i` inclus : `j = rand() % (i + 1)`.
 - (d) Echangez `deck->cards[i]` et `deck->cards[j]`.
4. Testez dans `test_deck.c` :
- (a) Affichez les 5 premieres cartes avant melange.
 - (b) Appelez `deck_shuffle(&deck)`.
 - (c) Affichez les 5 premieres cartes apres melange.
 - (d) Verifiez que l'ordre des cartes a change.

4.4 Exercice 1.4 : Pioche securisee

1. Creez la fonction suivante :

```
Card deck_draw(Deck *deck);
```

2. Dans cette fonction :
- (a) Verifiez si `deck->size == 0`.
 - (b) Si le deck est vide, retournez une carte speciale : type NUM, valeur -1.

- (c) N'affichez aucun message dans cette fonction (la gestion de l'erreur sera faite par l'appelant).
 - (d) Si le deck n'est pas vide, decrementez `deck->size` et retournez `deck->cards[deck->size]`.
3. Testez dans `test_deck.c` :
- (a) Creez un deck et initialisez-le.
 - (b) Utilisez une boucle pour piocher 86 fois.
 - (c) Verifiez que les 85 premieres pioches retournent des cartes valides.
 - (d) Verifiez que la 86eme pioche retourne la carte `{NUM, -1}`.
 - (e) Verifiez qu'il n'y a jamais de crash.

4.5 Exercice 1.5 : Fonctions utilitaires (optionnel mais recommande)

1. Creez une fonction `void deck_print(Deck *deck)` qui affiche toutes les cartes du deck. Cette fonction sera utile pour le debogage.
2. Creez une fonction `int deck_is_empty(Deck *deck)` qui retourne 1 si le deck est vide, 0 sinon.
3. Creez une fonction `int deck_size(Deck *deck)` qui retourne le nombre de cartes restantes.

5 Partie 2 : Moteur du jeu

Partie réservée à Kais

Cette partie est entièrement réservée à Kais. Les fichiers concernés sont : player.c, player.h, rules.c, rules.h, round.c, round.h, game.c, game.h.

5.1 Exercice 2.1 : Structure Player

1. Ouvrez le fichier player.h.
2. Déclarez la structure Player avec les champs suivants :
 - name : tableau de 50 caractères
 - hand : tableau de 20 Card
 - handSize : entier
 - roundScore : entier
 - totalScore : entier
 - bonusMulti : entier
 - bonusAdd : entier
3. Déclarez les prototypes des fonctions suivantes :

```
void player_init(Player *p, const char *name);
void player_add_card(Player *p, Card c);
int player_has_duplicate(Player *p, Card c);
int player_has_seven(Player *p);
void player_reset_round(Player *p);
```

4. Ouvrez le fichier player.c et implémentez ces fonctions :
 - (a) player_init :
 - Copiez le nom dans p->name avec strcpy.
 - Initialisez handSize = 0.
 - Initialisez roundScore = 0.
 - Initialisez totalScore = 0.
 - Initialisez bonusMulti = 1.
 - Initialisez bonusAdd = 0.
 - (b) player_add_card :
 - Stockez la carte dans p->hand[p->handSize].
 - Incrémentez p->handSize.
 - (c) player_has_duplicate :
 - Parcourez p->hand de 0 à p->handSize - 1.
 - Si une carte a le type NUM et la même valeur que c, retournez 1.
 - Sinon, retournez 0.
 - (d) player_has_seven :
 - Créez un tableau distinct[13] initialisé à 0.
 - Parcourez p->hand.
 - Pour chaque carte de type NUM, marquez distinct[value] = 1.
 - Comptez le nombre de valeurs distinctes.
 - Retournez 1 si le compte est ≥ 7 , sinon 0.
 - (e) player_reset_round :

- Remettez handSize = 0.
- Remettez roundScore = 0.
- Remettez bonusMulti = 1.
- Remettez bonusAdd = 0.
- Ne modifiez pas totalScore.

5.2 Exercice 2.2 : Application des regles

1. Ouvrez le fichier rules.h.
2. Creez le type enumere RoundStatus avec trois valeurs :

```
typedef enum {
    ROUND_CONTINUE,
    ROUND_LOST,
    ROUND_OVER_WIN
} RoundStatus;
```

3. Declarez les prototypes suivants :

```
RoundStatus rules_apply_card(Player *p, Card c);
int rules_compute_round_score(Player *p);
int rules_is_game_over(Player players[], int nbPlayers);
```

4. Ouvrez rules.c et implementez ces fonctions :

(a) rules_apply_card :

- Si la carte est invalide (type NUM et value == -1), retournez ROUND_CONTINUE.
- Si la carte est de type NUM :
 - Si player_has_duplicate(p, c) retourne 1, retournez ROUND_LOST.
 - Sinon, appelez player_add_card(p, c).
 - Ajoutez c.value a p->roundScore.
 - Si player_has_seven(p) retourne 1, retournez ROUND_OVER_WIN.
- Si la carte est de type BONUS_MULT :
 - Multipliez p->bonusMulti par c.value.
- Si la carte est de type BONUS_ADD :
 - Ajoutez c.value a p->bonusAdd.
- Retournez ROUND_CONTINUE par default.

(b) rules_compute_round_score :

- Calculez int score = p->roundScore.
- Multipliez score par p->bonusMulti.
- Ajoutez p->bonusAdd a score.
- Retournez score.

(c) rules_is_game_over :

- Parcourez le tableau des joueurs.
- Si un joueur a totalScore >= 200, retournez 1.
- Sinon, retournez 0.

5.3 Exercice 2.3 : Gestion des manches

1. Ouvrez le fichier round.h.
2. Declarez les prototypes suivants :

```
void round_reset_all_players(Player players[], int nbPlayers);  
int round_is_over(Player players[], int nbPlayers, int *winnerIndex);  
void round_apply_bonus_winner(Player *winner);
```

3. Ouvrez round.c et implementez ces fonctions :
 - (a) round_reset_all_players :
 - Parcourez tous les joueurs.
 - Pour chaque joueur, appelez player_reset_round.
 - (b) round_is_over :
 - Parcourez tous les joueurs.
 - Si player_has_seven(&players[i]) retourne 1 :
 - Stockez i dans *winnerIndex.
 - Retournez 1.
 - Retournez 0.
 - (c) round_apply_bonus_winner :
 - Ajoutez 15 a winner->totalScore.

5.4 Exercice 2.4 : Boucle de jeu principale

1. Ouvrez le fichier game.h.
2. Declarez le prototype suivant :

```
void game_run(Deck *deck, Player players[], int nbPlayers);
```

3. Ouvrez game.c et implementez game_run :
 - (a) Initialisez running = 1.
 - (b) Tant que running est vrai et que le deck n'est pas vide :
 - i. Reinitialisez tous les joueurs avec round_reset_all_players.
 - ii. Creez un tableau out[nbPlayers] initialise a 0.
 - iii. Initialisez roundRunning = 1.
 - iv. Tant que roundRunning est vrai :
 - A. Parcourez les joueurs dans l'ordre.
 - B. Si out[i] == 1, passez au joueur suivant.
 - C. Affichez l'etat du jeu (sera fait par Youssef).
 - D. Demandez au joueur s'il veut piocher ou sortir (sera fait par Youssef).
 - E. Si le joueur sort, out[i] = 1.
 - F. Si le joueur pioche :
 - G. Piochez une carte avec deck_draw.
 - H. Appliquez la regle avec rules_apply_card.
 - I. Si ROUND_LOST, out[i] = 1 et roundScore = 0.
 - J. Si ROUND_OVER_WIN, appliquez le bonus et roundRunning = 0.
 - v. Verifiez si tous les joueurs sont out.
 - vi. Calculez les scores finaux avec rules_compute_round_score.
 - vii. Ajoutez les scores aux totalScore des joueurs.
 - viii. Affichez le resume de la manche (sera fait par Youssef).
 - (c) Verifiez si la partie est terminee avec rules_is_game_over.

5.5 Exercice 2.5 : Intelligence artificielle (bonus)

1. Creez une fonction `int ai_decision(Player *p)` dans un fichier `ai.c`.
2. La strategie de l'IA doit etre :
 - Si `p->roundScore < 20`, retourner 1 (piocher).
 - Sinon, retourner 0 (sortir).
3. Modifiez `game_run` pour que les joueurs marques comme "IA" utilisent `ai_decision` automatiquement.

6 Partie 3 : Interface utilisateur

Partie réservée à Youssef

Cette partie est entièrement réservée à Youssef. Les fichiers concernés sont : display.c, display.h, input.c, input.h, save.c, save.h, stats.c, stats.h.

6.1 Exercice 3.1 : Affichage des cartes et des joueurs

1. Ouvrez le fichier display.h.
2. Déclarez les prototypes suivants :

```
void display_card(Card c);
void display_player(const Player *p);
void display_game_status(const Player players[], int currentPlayer,
    int nbPlayers, const Deck *deck);
void display_round_summary(const Player players[], int nbPlayers);
void display_winner(const Player players[], int nbPlayers);
```

3. Ouvrez display.c et définissez les macros de couleurs ANSI :

```
#define RED      "\033[31m"
#define GREEN    "\033[32m"
#define YELLOW   "\033[33m"
#define BOLD     "\033[1m"
#define RESET    "\033[0m"
```

4. Implémentez display_card :
 - Si le type est NUM, affichez [valeur].
 - Si le type est BONUS_MULT, affichez en VERT [x valeur].
 - Si le type est BONUS_ADD, affichez en ROUGE [+ valeur].
5. Implémentez display_player :
 - Affichez le nom du joueur.
 - Affichez "Cartes en main : ".
 - Parcourez la main du joueur et appelez display_card pour chaque carte.
 - Affichez "Score de la manche : X" et "Score total : Y".
6. Implémentez display_game_status :
 - Affichez un séparateur visible (par exemple 50 fois le caractère '-').
 - Affichez "Joueur actuel : NOM" en gras.
 - Affichez "Cartes restantes dans la pioche : X".
 - Appelez display_player pour le joueur courant.
7. Implémentez display_round_summary :
 - Affichez "Fin de la manche - Scores :".
 - Pour chaque joueur, affichez son nom et son score de manche.
8. Implémentez display_winner :
 - Trouvez le joueur avec le score total le plus élevé.
 - Affichez "Le gagnant de la partie est NOM avec X points".

6.2 Exercice 3.2 : Saisie securisee

1. Ouvrez le fichier input.h.
2. Declarez les prototypes suivants :

```
int safe_input_int(const char *prompt, int min, int max);
void safe_input_string(char *buffer, int size, const char *prompt);
int safe_input_yes_no(const char *prompt);
int safe_input_choice_pipick(const char *prompt);
```

3. Ouvrez input.c et implementez safe_input_int :
 - Affichez le prompt.
 - Lisez un entier avec scanf.
 - Si la lecture echoue (resultat != 1), videz le buffer avec while(getchar() != '\n').
 - Si la valeur est comprise entre min et max, retournez-la.
 - Sinon, affichez une erreur et recommencez.
4. Implementez safe_input_string :
 - Affichez le prompt.
 - Lisez avec fgets(buffer, size, stdin).
 - Supprimez le caractere '\n' final si present.
5. Implementez safe_input_yes_no :
 - Affichez le prompt.
 - Lisez une chaine avec safe_input_string.
 - Si la chaine est "o", "O", "oui" ou "OUI", retournez 1.
 - Si la chaine est "n", "N", "non" ou "NON", retournez 0.
 - Sinon, affichez une erreur et recommencez.
6. Implementez safe_input_choice_pipick :
 - Affichez "Piocher (1) ou sortir (2) ?".
 - Appelez safe_input_int avec min=1 et max=2.
 - Retournez la valeur.

6.3 Exercice 3.3 : Sauvegarde des scores

1. Ouvrez le fichier save.h.
2. Declarez les prototypes suivants :

```
void save_scores_to_file(const Player players[], int nbPlayers);
void load_scores_from_file(const char *filename);
```

3. Ouvrez save.c et implementez save_scores_to_file :
 - Declarez un tableau char filename[100].
 - Demandez le nom du fichier avec safe_input_string.
 - Ouvrez le fichier avec fopen(filename, "w").
 - Si l'ouverture echoue, affichez une erreur et retournez.
 - Pour chaque joueur, ecrivez dans le fichier : "NOM | SCORE_TOTAL | SCORE_MANCHE\n".
 - Fermez le fichier avec fclose.
 - Affichez un message de confirmation.
4. Implementez load_scores_from_file (optionnel) :
 - Ouvrez le fichier en lecture.
 - Lisez ligne par ligne avec fgets.
 - Affichez le contenu.
 - Fermez le fichier.

6.4 Exercice 3.4 : Statistiques

1. Ouvrez le fichier stats.h.
2. Declarez les prototypes suivants :

```
void stats_display_best_player(const Player players[], int nbPlayers);  
void stats_display_average_scores(const Player players[], int  
    nbPlayers);  
void stats_display_all_scores(const Player players[], int nbPlayers);
```

3. Ouvrez stats.c et implementez stats_display_best_player :
 - Parcourez le tableau des joueurs.
 - Trouvez l'indice du joueur avec le totalScore le plus eleve.
 - Affichez "Meilleur joueur : NOM avec X points".
4. Implementez stats_display_average_scores :
 - Calculez la somme de tous les totalScore.
 - Divisez par nbPlayers pour obtenir la moyenne.
 - Affichez "Moyenne des scores : X.YZ".
5. Implementez stats_display_all_scores :
 - Pour chaque joueur, affichez son nom et son score total.
 - Formatez l'affichage en colonnes alignees.

6.5 Exercice 3.5 : Cartes speciales (bonus)

1. Ajoutez les types de cartes supplementaires dans CardType :
 - BONUS_STOP
 - BONUS_SECONDE_CHANCE
 - BONUS_TROIS_A_LA_SUITE
2. Ajoutez les 3 cartes speciales dans deck_init.
3. Modifiez rules_apply_card pour gerer ces nouveaux types.

7 Partie 4 : Integration, tests et rendu final

7.1 Exercice 4.1 : Programme principal

1. Ouvrez le fichier main.c.
2. Incluez tous les fichiers d'en-tete :

```
#include "deck.h"
#include "player.h"
#include "rules.h"
#include "round.h"
#include "display.h"
#include "input.h"
#include "save.h"
#include "stats.h"
```

3. Ecrivez la fonction main :
 - (a) Declarez Deck deck et initialisez-le avec deck_init.
 - (b) Melangez le deck avec deck_shuffle.
 - (c) Demandez le nombre de joueurs avec safe_input_int (min=1, max=10).
 - (d) Declarez un tableau de joueurs : Player players[nbPlayers].
 - (e) Pour chaque joueur, demandez son nom avec safe_input_string et appelez player_init.
 - (f) Lancez la boucle de jeu avec game_run(&deck, players, nbPlayers).
 - (g) Affichez le gagnant avec display_winner.
 - (h) Proposez d'enregistrer les scores avec safe_input_yes_no.
 - (i) Si l'utilisateur accepte, appelez save_scores_to_file.
 - (j) Proposez d'afficher les statistiques.
 - (k) Retournez 0.

7.2 Exercice 4.2 : Liste des tests obligatoires

Avant de rendre le projet, verifiez que tous les cas suivants fonctionnent :

Cas a tester	Resultat attendu
Piocher une carte numero nouvelle	La carte s'ajoute a la main
Piocher un doublon	Perte de la manche (score = 0)
Piocher un bonus x2	Le score final est multiplie par 2
Piocher un bonus +X	Le score final est augmente de X
Obtenir 7 cartes differentes	Fin manche + 15 points pour le joueur
Pioche vide	Fin de partie normale
Score atteint 200 points	Victoire immediate
Saisie d'une lettre au lieu d'un nombre	Le programme redemande sans planter
Saisie d'un nombre hors bornes	Le programme redemande
Sauvegarde des scores	Un fichier est cree avec le bon format

7.3 Exercice 4.3 : Contrôles de qualité finaux

Vérifiez les points suivants avant le rendu :

- ✓ Le programme compile avec la commande make.
- ✓ Le programme ne crash jamais.
- ✓ Aucune variable globale n'est utilisée.
- ✓ Chaque fonction fait moins de 40 lignes.
- ✓ Chaque fichier fait moins de 300 lignes.
- ✓ Le code est commenté (anglais recommandé).
- ✓ La logique métier et l'affichage sont séparés.
- ✓ Toutes les erreurs utilisateur sont gérées.
- ✓ Le Makefile fonctionne correctement.

7.4 Contenu du livrable

- URL du dépôt Git (public, accessible par le chargé de TD)
- Code source complet (arborescence conforme à la section 3)
- Makefile (la commande make doit compiler tout le projet)
- README.md expliquant comment compiler et exécuter le jeu
- Rapport PDF de 3 à 5 pages contenant :
 - La description de l'équipe et du sujet
 - L'organisation du travail (répartition, outils, communication)
 - Les difficultés rencontrées et les solutions apportées
 - Les choix de conception (pourquoi cette architecture ?)
 - Les fonctionnalités implémentées et les bonus

7.5 Barème indicatif

Critère	Points
Partie 1 – Deck (Brahim)	4
Partie 2 – Moteur (Kais)	5
Partie 3 – Interface (Youssef)	4
Stabilité (aucun crash)	4
Bonus (minimum 2)	3
Total	20

7.6 Conseil final

Attention

Un projet qui crash est éliminé. Il est préféré d'avoir 80 pourcents des fonctionnalités avec un programme stable, plutôt que 100 pourcents des fonctionnalités avec un programme qui plante.